

AI-Native Knowledge Compiler (`akb`)

The persistent, auditable memory layer for coding agents. coding agent 的可审计持久记忆层。

⚠️ 状态：设计阶段（**design-stage**）。本仓库目前只有设计文档，没有可运行代码。下面的路线图、命令、数据结构都是规约，不是已实现的功能。

这是什么

`akb` 是一个 **git-backed**、**markdown-native**、**MCP-first** 的知识库系统，目标是让 coding agent（Claude Code / Codex / Cursor 等）能够**长期、稳定、可追溯地依赖**一份工程知识——而这份知识同时是人类可读、可编辑、可 PR review、可 `git blame` 的。

一句话定位：人和 **agent** 共享同一份事实。

它不是：

- 不是另一个 Notion / Confluence —— 那些不为 agent 设计，也不可 `git`
- 不是另一个 Obsidian —— 那个是给人用的，agent 接口靠插件凑
- 不是另一个 RAG framework —— Dify / RAGFlow 是 query-time 检索，`akb` 是 ingest-time 编译
- 不是 agent 的黑盒 cache —— Devin / Cursor 的内部 memory 不可读、不可改、锁在 vendor 里

它是：

- 一个 git-backed markdown vault，知识以纯文本累积、演化、版本化
- 一个 MCP-first 接口，任何支持 MCP 的 agent 都能查同一个知识库
- 一个有 **confidence 演化机制**的“知识 git”——不仅记录“现在认为什么”，也记录“过去认为过什么、为什么改了”

为什么从头写——以及与现有方案的差异

Karpathy 在 2026 年 4 月提出的 [LLM Wiki 模式](#) 已经有多个社区实现（[NiharShrotri/llm-](#)

wiki、nashsu/llm_wiki、nvk/llm-wiki、ΩmegaWiki 等）。我们认真调研过这些实现，结论是：“**ingest-time compile 而非 query-time RAG**”这个核心范式已经被反复实现，不值得再造。但有七项能力，没有任何现有实现覆盖——而这七项恰好是“让 agent 长期可信地依赖知识库”的关键。

Prior Art 对照

能力	现有 LLM Wiki 实现	akb
ingest-time compile（非 query-time RAG）	✅ 已是标配	✅ 采用
merge / create / contradict 关系判定	✅ 已是标配	✅ 采用
Obsidian 兼容 / <code>[[wikilink]]</code>	✅ 已是标配	✅ 采用
混合检索 BM25 + vector + rerank	✅ 部分实现	✅ 采用
MCP 接口（agent-native）	❌ 无	✅ 核心差异
行号级 citation （page_id + 行号范围）	⚠️ 仅页级	✅ 核心差异
Confidence Ledger （append-only 事件流）	⚠️ 仅 LLM 主观打分	✅ 核心差异
Chunk Lineage （derived chunk 可溯源 + replay）	⚠️ 仅页级 / session 级	✅ 核心差异
Runtime Verification （CI / runbook 自动验证知识）	❌ 无	✅ 核心差异
内建 Eval 框架（每个 PR 跑 golden set 回归）	❌ 无	✅ 核心差异
Multi-Agent 协作（签名、配额、handoff、peer review）	❌ 无	✅ 核心差异
代码反向解析（codebase → 设计文档）	❌ 无（面向 research/PKM）	✅ 核心差异（规划中）

中期对标：**Karpathy LLM Wiki 模式的工程化实现**——同时具备 MCP 协议、git 原生、citation、confidence、code intel。这是目前还没人把所有零件拼齐的位置。

长期定位：coding agent 的工程记忆层。但**不是要做更好的 Devin memory**，而是做相反方向——human-readable + git-versioned + multi-agent-shared。两者长期会并存（agent 自己的快记忆 + 团队的慢记忆），akb 占的是后者。

核心设计原则

1. Markdown 是 canonical，其它都是投影

markdown 是事实源 (canonical truth)。所有 index / vector / graph / confidence 事件缓存都是从 markdown 派生的投影层 (projection layer)，永远不反过来。

- 任何投影任何时候都可以从 markdown 完全重建
- 投影与 markdown 冲突时，markdown 赢
- 任何写操作必须先写 markdown，再更新投影
- 投影层不入 git，只有 markdown 和事件流 (JSONL) 入 git

我们明确拒绝 **claim-first / graph-first 架构** (让外部 graph 是真相、markdown 是 render layer)。理由：LLM 抽 claim 准确率长尾差、人类无法直接编辑、叙事被切碎、双向同步是分布式系统级难题。详见 [v0.0 spec § 0.6](#)。

2. Citation-first

没有 citation 的知识系统最终会退化成“模型觉得”而不是“知识库事实”。akb 强制每个检索结果带可定位的 citation——精确到 `page_id + line_start + line_end`。patch、supersede、contradiction、confidence、replay 全部建立在 knowledge provenance 之上。

3. Compile-time vs Query-time 的工作划分

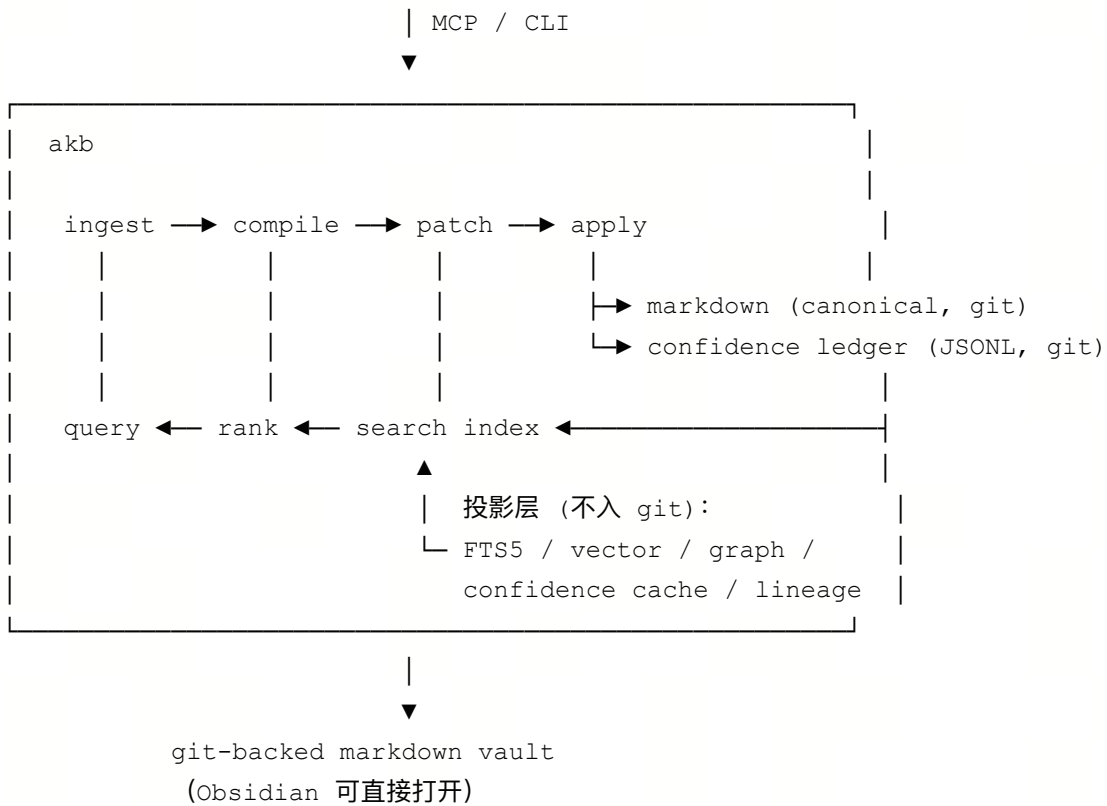
产品名字里 “Compiler” 的技术兑现：昂贵的、累积的、需要 LLM 判断的工作发生在 **ingest time (compile time)**，只做一次；**query time** 只剩“检索 + 重排 + 取内容”这种廉价、无状态、确定的操作。这是与 RAG 的本质区别——RAG 把组织工作放 query time 每次重做。

4. Confidence 是事件流，不是分数

Confidence Ledger 不是一个 `confidence: 0.82` 字段，而是一个 **append-only 事件流**——知识库的 git log。知识不应该被覆盖，而应该演化。Page 的 confidence 是这个事件流的物化视图。它解决的是普通 RAG 系统一定会遇到的退化：claim 永远新鲜、矛盾静默共存、来源无权重、老知识不衰减。详见 [v0.1 confidence ledger](#)。

架构概览

人 / coding agent



- **canonical:** `pages/*.md + .ledger.jsonl` — 入 git, 是事实
- **projection:** SQLite FTS5、vector index、relation graph、confidence cache、chunk lineage — 不入 git, 可随时从 canonical 重建
- **接口:** CLI (`akb ...`) + MCP server (`search_knowledge / get_page / ...`)

路线图

工期估算基于单人专注。“业余”按 ×2 估。每个版本都要求端到端可用，不接受半成品瀑布。

v0.0 — 跑通最小闭环 (~4 周)

证明 `ingest → index → MCP → coding agent retrieval → citation` 这条管道可靠。

- `akb init / ingest / index / search / mcp serve / eval`
- git-backed markdown vault, Obsidian 兼容
- SQLite FTS5 + BM25, 行号级 citation
- 2 个 MCP tool: `search_knowledge`、`get_page`
- 内建 eval 框架, 每个 PR 跑 golden set 回归

详见 [docs/v0.0-spec.md](#) 和 [docs/search-engine-skeleton.md](#)。

v0.1 — 知识不腐烂 + 知识自己长在一起 (~3-4 个月)

- **Confidence Ledger** (~21 天) — append-only 事件流、时间衰减、来源权重、supersession 链、confidence-aware retrieval、runtime verification。详见 [docs/v0.1-confidence-ledger.md](#)
- **LLM Compile** (~27 天) — 5 阶段 pipeline、关系判定、patch-as-proposal、chunk lineage、replay。详见 [docs/v0.1-llm-compile.md](#)
- Patch workflow、vector search、hybrid retrieval、`akb ask`

实施顺序：confidence ledger 先 (compile 的 patch 要写 ledger 事件) → compile → vector + ask。

v0.2 — Multi-Agent 协作 (~5-7 周)

让多个 agent 能同时演化同一份知识、不打架、可追溯、能审计、能撤销。直面 git 不解决的四个问题：并发协调、身份信任、速率控制、选择性撤销。

- **Layer 1 并发写入**——Patch 显式 base commit、四态 apply 决策、自动 re-compile on reject、`akb push/pull` 包装
- **Layer 2 Agent 身份与信任**——registry、ed25519 签名、配额、audit log、影响撤销 (不删历史)
- **Layer 3 协作语义**——peer review (不同 agent 互审)、handoff ledger (结构化任务交接)、specialty 路由
- **Layer 4 并发 read**——freshness 透明化、scope 限定、可选 trace
- **Layer 5 共享知识库**——vault federation (git submodule 模型)、branch workflow、per-page ACL

详见 [docs/v0.2-multi-agent.md](#)。

v0.3 及以后

- Section-level confidence (按 markdown header 切分的中间粒度)
 - Code intelligence — codebase 反向解析成设计文档 / ADR
 - GraphRAG、Web UI、bidirectional federation
-

设计文档索引

文档	内容
docs/v0.0-spec.md	v0.0 极简 spec —— 最小闭环、12 个可执行 issue、定位与 prior art、投影层原则
docs/search-engine-skeleton.md	search-engine 包的代码骨架 —— API / SQL schema / 测试用例 frozen，可直接交给 agent 实现
docs/v0.1-confidence-ledger.md	Confidence Ledger 设计 —— 事件流、衰减公式、来源权重、supersession、runtime verification
docs/v0.1-llm-compile.md	LLM Compile 设计 —— 5 阶段 pipeline、chunk lineage schema、与 confidence ledger 的集成
docs/v0.2-multi-agent.md	Multi-Agent 协作设计 —— 五层架构、签名/配额/审计、peer review、handoff ledger、vault federation

建议阅读顺序： v0.0-spec → confidence-ledger → llm-compile → multi-agent → search-engine-skeleton 。

关键设计纪律

写在最前面，避免 scope creep：

1. 任何要 LLM API key 的功能都不在 v0.0 —— 保证 v0.0 跑通不需要任何外部账号
2. 任何让 LLM 写入 vault 的功能都走 patch + review gate，不直接写
3. 任何加新 MCP tool 的提议都要先证明现有的不够 —— 工具爆炸是 agent 系统的头号风险
4. 任何投影层数据都不入 git —— git 只追事实源
5. 任何“修改 vault”的代码路径都必须先写 markdown 再更新投影
6. 每个 PR 必须跑通 eval，retrieval 准确率不允许回归
7. confidence 由系统事件自动产生，不允许 LLM 给自己打分 —— 那样无法 audit

Prior Art & 致谢

- [Andrej Karpathy](#) — [LLM Wiki 模式](#)，本项目的范式来源
- [NiharShrotri/llm-wiki](#) — 完成度最高的 CLI 实现，3-pass ingest + 混合检索 + lint 的工程参考
- [nvk/llm-wiki](#) — agent-native 的 AGENTS.md 协议、structural guardian、dual-linking 格式参考
- [nashsu/llm_wiki](#) — two-step CoT ingest、4-signal 知识图谱的设计参考
- **OmegaWiki / llm-wiki KG 实现** — edge-level provenance + `needs_review` flag 的设计印证

akb 站在这些工作之上，补的是它们共同缺失的那一块：**让知识库成为 agent 可长期信任、人可持续审计的工程记忆。**

License

MIT — 见 [LICENSE](#)。

贡献

当前处于设计阶段。如果你对设计文档有意见、发现 prior art 里我们漏掉的实现、或想认领某个 v0.0 issue 的实现，欢迎开 issue 讨论。

代码开始前，所有架构变更先走 issue + 设计文档 PR，不直接写代码——这个项目本身就是在实践“知识先于代码”。